

DODVins

A Data-Oriented MSCKF Pipeline for Predictable Visual-Inertial Odometry
project walkthrough — what we built, how we measured it, what it buys

Shafwan Safi Nikhil Vashisht Pranjal Upadhyay

IISER Bhopal

The question this project answers

- VIO runs **per frame, forever**, on hardware that also drives control and navigation.
- OpenVINS is the reference MSCKF implementation — but it is built for extensibility: heap-allocated per-feature objects, `std::vector` churn, virtual dispatch.
- **Open question:** on a production MSCKF pipeline, how much of the per-frame cost is the algorithm, and how much is the implementation pattern?

DODVins = OpenVINS's estimator, re-implemented under data-oriented design constraints, measured against the original.

The honest scope, up front

What is unchanged

The estimator's **model**: same MSCKF equations, same state, same measurement model as OpenVINS.

What differs — and we classify every change

- **Storage/layout**: fixed-capacity state, feature database separating 4-byte keys from payloads, no virtual dispatch.
- **Solver restructures** (numerically equivalent or accuracy-gated, each measured separately): square-root-form EKF update, one-GEMM M_a , measurement compression, χ^2 early-accept.
- **Initializer**: feature-less solve (from sqrtVINS) — this is where part of the accuracy-difference story comes from.

House rules for every line of frame-path code

- ① No variable-sized per-frame objects (no `new` for features, measurements, state; no `push_back` churn).
- ② Every capacity fixed at init and **asserted** — overflow aborts loudly, never corrupts.
- ③ No virtual dispatch, no RTTI on the hot path.
- ④ Existence-based processing: membership in a pre-filtered table is the branch.
- ⑤ Transforms are pure: read inputs, write outputs, no read-back.

Every optimization was accepted only after DODVins's ten trajectories stayed **bit-identical to its own pre-change output**.

How we measured — the rules that make the numbers checkable

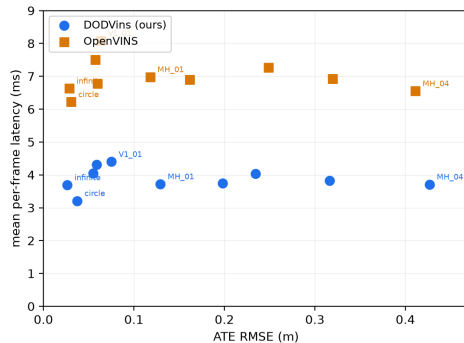
- **Same transport:** both estimators read the identical rosbag.
- **Same toolchain:** identical Docker container (Ubuntu 20.04, ROS1 Noetic, OpenCV 4.2.0, g++ 9.4.0) on both sides — and on the Jetson.
- **Control-tree gate:** two commits built side by side, run on identical data, estimate CSVs compared **byte-for-byte**.
- **Profile, not guess:** two plausible speed hypotheses were tested and found wrong; both are reported.

We also measured two confounds that invalidate casually-quoted ATEs: **toolchain choice moves ATE by up to 23%** (identical code, identical data, different OpenCV/g++), transport by 0–5.2%.

Result 1 — accuracy: parity, not superiority

	DODVins	OpenVINS
Mean ATE (10 seq)	0.156 m	0.158 m
Better on	4 of 10 sequences	
Drift (1 s RPE)	better or equal on 7 of 8 EuRoC	

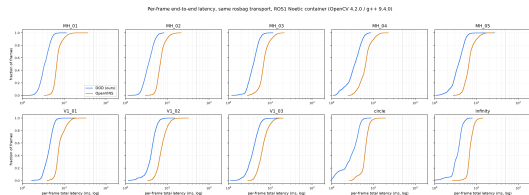
- MH_02's gap (0.198 vs 0.162) is an **initialization transient**: DODVins initializes ~ 2 s earlier from a worse state; drift is nearly identical.
- RPE — not ATE — reveals this. ATE alone would misattribute it to filter quality.



Result 2 — speed: consistent, on both axes

	Wall (s)		CPU (s)	
	DOD	OV	DOD	OV
MH_01	14.0	29.5	29.5	44.9
MH_04	7.7	15.7	16.3	24.3
circle	15.7	27.6	32.5	42.8

1.76–2.10 $\times$ wall-clock; 24–34% less total processor time — the quantity a duty-cycled platform pays for.



Result 3 — the tail: predictability, our strongest number

- p99 frame-latency advantage: **1.64–2.05** $\times$ — *exceeds* the mean advantage (1.63–1.85 $\times$) on 8 of 10 sequences.
- Worst observed frame: DODVins 8–18 ms vs. OpenVINS 20–55 ms. OpenVINS's worst (KAIST circle, 54.8 ms) blows the 16 Hz frame deadline; DODVins never exceeds a third of any budget.

Why the tail matters

Allocator activity and cache misses land in spikes. Winning *more* in the tail than in the mean is the signature of a layout win, not an arithmetic win.

Where the speed comes from — classified, not assumed

Solver restructures: 1.22 ms/frame attributable (χ^2 gate, sqrt-form, GEMM, compression).

Storage/layout: 0.24 ms/frame + feature-DB memcpy elimination (9,556 copies/frame $\rightarrow$ 0).

Mixed (row 8, 1.97 ms): arithmetic and placement restructured together — cannot be split honestly.

Honest scaling example

The square-root EKF update:
0.29 ms/frame ($\approx 8\%$ of the total gain).
Its primary contribution is **correctness** — the dense form drives covariance diagonals negative and aborts MH_01 at $t=16$ s.

Every row of the ablation was measured with all ten trajectories bit-identical before and after.

"Allocation-free" — what we claim, precisely

DHAT attribution, 12 s of MH_01 (777,007 blocks, 1.89 GB):

- 59% — OpenCV KLT tracker internals (not ours)
- 19% — DODVins estimator: **bounded Eigen workspaces**, 137 blocks / 3.8 MB per frame in EKFUpdate
- 15% — one-time library startup; 3% — dataset I/O

Precise claims

- Propagation path: **zero** allocations (unit-test asserted).
- No per-feature / per-measurement heap objects, ever.
- Working set does not grow with sequence length.

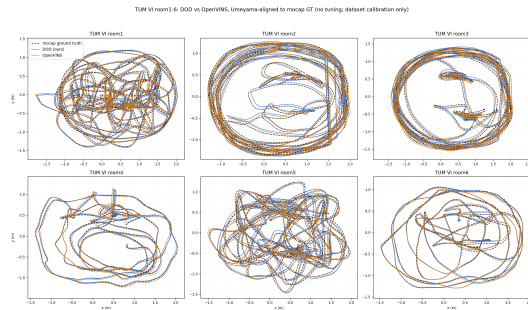
We do *not* claim the dense update workspaces are allocation-free — we measured them instead.

Result 4 — Jetson Orin Nano: it transfers, narrower

- Same toolchain container on aarch64 — **ISA is the only variable**.
- Median latency: 27–35 ms vs OpenVINS 34–47 ms (**1.16–1.61×**); faster on all ten sequences.
- Margin *narrows*: p99 advantage is smaller than on x86 (plausible cause: LPDDR5 bandwidth — untested, stated as such).
- Peak RSS: **184–199 MB flat** vs OpenVINS 98–179 MB variable. We trade higher average utilization for a bounded working set — owned, not hidden.

Result 5 — TUM VI: the untuned independent test

- Third dataset family, first fisheye camera (Kannala-Brandt 4 — the pipeline already had the model).
- **Only calibration adapted**; every estimator knob is the marked EuRoC placeholder.
- All six room sequences initialize and converge.
- Mean ATE **0.062 vs 0.063 m** (OpenVINS, same Kalibr release): parity, DODVins better on 3 of 6.



Result 6 — high-altitude outdoor: where MSCKF breaks

Alt	DODVins ($\sigma=1$)	tuned σ	OpenVINS
40 m	12.81	12.81	4.19
60 m	1367.7 $\times$	48.3	12.22
80 m	56303 $\times$	23.2	145059 $\times$
100 m	94897 $\times$	110.3	57.2

Outdoor UAV, 30 cm baseline, 40–100 m altitude: depth/baseline ratio 130–330.

- **No shipped config covers the envelope** — both pipelines are one inherited constant from divergence.
- DODVins latency outdoors unchanged: 2.6–3.9 ms median.

What this does not claim

- **No estimator novelty** — every equation is OpenVINS's; the contribution is implementation + measurement.
- **Accuracy superiority** — parity; worse on 6 of 10.
- **Lower memory** — higher average RSS; the claim is *bounded*, not minimal.
- **Full allocation-freedom** — propagation is; the update's dense workspaces are measured, not hidden.
- Untuned-on-TUM-VI means the numbers there are the inherited configuration's, not the pipeline's best.
- Jetson thermal/frequency state uncontrolled (limitations say so).

- Desktop x86: same MSCKF trajectories for $\sim$ half the wall-clock and a quarter to a third less processor time, with a frame-latency tail inside the sensor's budget.
- Jetson-class SoC: same directions, narrower margins.
- Estimator must share compute with control/navigation? DODVins frees that share **without changing a filter equation**.
- Relying on loop closure or tight memory ceilings? Weigh the recorded trade-offs first.
- And regardless of pipeline: quote ATEs with transport *and* toolchain, or they cannot be reproduced.

- Public repo: `github.com/Shafwansafi06/vio_pipeline_cpp` (tag `v0.1.0` = submission snapshot).
- `artifact/README.md`: every paper number $\rightarrow$ its archived run or its script.
- Harnesses included: x86 latency/RSS matrix, Orin matrix, control-tree byte-compare (the method behind every optimization claim).
- Paper: `paper/main.pdf` — 8 pages, every number traced.

- ① Same MSCKF math, data-oriented implementation: **1.76–2.10× faster** at accuracy parity.
- ② The tail (p99) improves *more* than the mean — the layout signature.
- ③ Every optimization individually measured, classified (solver vs storage), and gated bit-identically.
- ④ Transfers to Jetson (narrower), untuned to TUM VI (parity), honestly broken at high altitude like everything else — with the failure diagnosed by counters, not guesses.
- ⑤ Two benchmarking confounds (toolchain 23%, transport 5%) measured, so our numbers — and anyone else's — can be reproduced.

Questions welcome — every number has a ticket behind it.